

Keywords, Variables, and Data Types

1. Python Keywords

- Keywords are special reserved words that are part of the Python language itself.
- They define the fundamental rules and structure of your programs.
- Because they have built-in meanings, you cannot use them as names for your variables, functions, or classes.
- If you attempt to use a keyword as a variable name, Python will immediately raise a `SyntaxError`.
- **Examples:** `True`, `False`, `None` (Value keywords), `and`, `or`, `not` (Operator keywords), and `if`, `for`, `while` (Control Flow keywords).

2. Python Variables

- Variables are used to store data that can be referenced and manipulated during program execution.
- A variable is essentially a custom name that is assigned to a value.
- **Dynamic Typing:** Unlike many other languages, Python variables do not require explicit declaration of their type; the type is automatically inferred based on the value assigned.
- **Object Reference:** Python variables store references to objects in memory, not the actual values themselves. When you reassign a variable, it simply references a new object.

2.1 Rules for Naming Variables

To avoid errors and write valid code, your variable names must follow these strict rules:

1. Names can only contain letters, digits, and underscores (`_`).
2. The first character of a variable name cannot be a digit.

- Names are strictly case-sensitive (e.g., myVar and myvar are treated as completely different variables).
- You cannot use Python keywords (like if, for, or else) as variable names.

2.2 Valid vs. Invalid Variable Names

Valid Examples:

```
Python
user_age = 25
_hidden_value = 10
player2_score = 100
```

Invalid Examples & Errors:

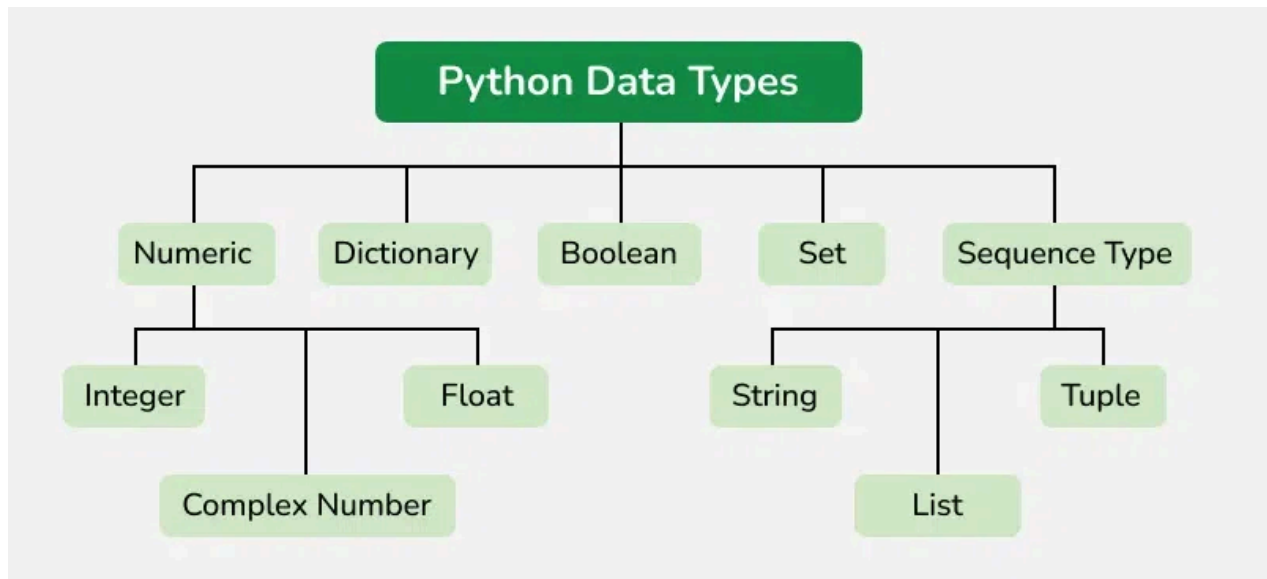
```
Python
# Starts with a digit
1st_place = "Gold" # Throws: SyntaxError: invalid decimal literal

# Uses a Python keyword
for = 10 # Throws: SyntaxError: invalid syntax

# Contains special characters other than an underscore
my-name = "Alex" #Throws: SyntaxError: cannot assign to
expression Maybe you meant '==' instead of '='?
```

3. Python Data Types

- Data types define the type of value stored in a variable, which helps the interpreter understand how to process the data.
- Python treats everything as an object.
- This means that every data type in Python is actually a **class** under the hood, and the variables you create are **objects** (or instances) of those classes.



(Figure from geeksforgeeks)

3.1 The type() Function

Because variables are objects, you can easily check which class a variable belongs to using the built-in `type()` function.

```
Python
x = 10
print(type(x)) # Output: <class 'int'>
```

```
y = 3.14
print(type(y)) # Output: <class 'float'>
```

3.2 Numeric Types

Numeric data types store mathematical values.

- **Integer (int):** Positive or negative integers without decimals.

```
Python
age = 25
```

- **Float (float):** Real numbers with a floating-point representation (a decimal point).

```
Python
price = 19.99
```

- **Complex (complex):** Numbers with real and imaginary parts.

```
Python
z = 2 + 3j
```

3.3 Sequence Types

A sequence is an ordered collection of items.

- **String (str):** Used to store text data. It can be enclosed in single, double, or triple quotes.

Python

```
name = "Alice"
```

- **List (list):** An ordered, mutable (changeable) collection of items, written with square brackets.

Python

```
fruits = ["apple", "banana", "cherry"]
```

- **Tuple (tuple):** An ordered, immutable (unchangeable) collection of items, written with round parentheses.

Python

```
coordinates = (10, 20)
```

3.4 Other Important Types

- **Boolean (bool):** Represents either True or False, mainly used in conditions.

Python

```
is_active = True
```

- **Set (set):** An unordered, mutable collection used to store unique elements.

Python

```
unique_ids = {1, 2, 3, 4}
```

- **Dictionary (dict):** Stores data in key:value pairs. Each key must be unique.

Python

```
user = {"name": "Bob", "age": 30}
```

4. Type Conversion

Type Conversion is converting a variable from one data type to another. Python handles this in two ways:

- Implicit (automatic)
- Explicit(Manual)

4.1 Implicit Type Conversion

- Python automatically converts one data type to another without user intervention.
- This happens to prevent data loss.
- Python only converts numeric types (bool, int, float, complex) automatically during math operations.
- It does this by upgrading the smaller data type to the larger, more precise data type to avoid losing details.
- Python will not automatically convert non-numeric types (like strings, lists, or tuples) when they meet other data types. Instead, it throws a `TypeError`.

4.1.1 Complete Implicit Conversion Table

This table shows exactly what data type you get when **Type A** meets **Type B** in a standard math operations (like +, -, or *).

Type A	Type B	The Result Type	Quick Example
bool (Boolean)	bool (Boolean)	int	True + True becomes 2

bool (Boolean)	int (Integer)	int	True + 5 becomes 6
bool (Boolean)	float (Float)	float	True + 2.5 becomes 3.5
bool (Boolean)	complex (Complex)	complex	True + 1j becomes (1+1j)
int (Integer)	int (Integer)	int	4 + 2 becomes 6
int (Integer)	float (Float)	float	4 + 2.0 becomes 6.0
int (Integer)	complex (Complex)	complex	4 + 3j becomes (4+3j)
float (Float)	float (Float)	float	1.5 + 2.5 becomes 4.0
float (Float)	complex (Complex)	complex	1.5 + 2j becomes (1.5+2j)
complex (Complex)	complex (Complex)	complex	1j + 2j becomes 3j

4.1.2 Strict Rules: What Python Refuses to Convert

- Python values data safety.
- If you try to mix numbers with text or collections, Python **will not** convert them. It will crash and give you a `TypeError`.
- **`str + int` → `TypeError`**

Example

Python

```
"Age: " + 25 # Raises TypeError
```

- **`str + float` → `TypeError`**

Example

Python

```
# Example: Mixing string and float  
"Price: " + 19.99 # Raises TypeError
```

- **`list + tuple` → `TypeError`**

Example

Python

```
# Example: Mixing different collection types  
[1, 2] + (3, 4) # Raises TypeError
```


Example: Automatic Integer to Float Conversion

Python

```
# Python automatically converts an integer to a float when  
performing operations with a float.
```

```
num_int = 123 # Integer value
```

```
num_flo = 1.23 # Float value
```

```
# New value will be a float
```

```
num_new = num_int + num_flo
```

```
print("Value:", num_new)
```

```
print("Data type:", type(num_new))
```

```
# Output
```

```
Value: 124.23
```

```
Data type: <class 'float'>
```

4.2 Explicit Type Conversion

Explicit type conversion is when a user manually changes the data type using built-in functions.

4.2.1. Conversion to Integer: int()

- Converts compatible data types (like floats or string numbers) into integers.
- Floats are truncated, meaning the decimal part is chopped off.

Example

Python

```
# Converting float to int
```

```
print(int(10.75)) # Output: 10
# Converting string numeric to int
print(int("45")) # Output: 45
# Error Case: Cannot convert non-numeric strings
# print(int("apple")) # Raises ValueError
```

4.2.2 Conversion to Float: float()

Converts integers or compatible string numbers into floating-point numbers.

Example

```
Python
# Converting int to float
print(float(25)) # Output: 25.0
# Converting string numeric to float
print(float("3.14")) # Output: 3.14
```

4.2.3 Conversion to String: str()

- Converts any data type (numbers, lists, tuples) into its string representation.
- This is highly useful for concatenating numbers with text.

Example

```
Python
# Converting int to string
age = 21
print("Age is " + str(age)) # Output: Age is 21
# Converting list to string
```

```
my_list = [1, 2, 3]
print(str(my_list))           # Output: "[1, 2, 3]" (as text)
```

4.2.4 Conversion to Boolean: bool()

Converts values to True or False.

- **False values:** 0, 0.0, empty strings "", empty containers [], (), {}, and None.
- **True values:** Any non-zero number or non-empty object.

Example

```
Python
# Evaluates to False
print(bool(0))           # Output: False
print(bool(""))          # Output: False
# Evaluates to True
print(bool(42))          # Output: True
print(bool("Hello"))     # Output: True
```

4.3 Collection Conversions: list(), tuple(), set()

You can freely convert between different sequence types to change how the data behaves (e.g., making it mutable or removing duplicates).

Example

```
Python
# Start with a tuple (immutable)
my_tuple = (1, 2, 2, 3)
# Convert to list (mutable, allows modifications)
```

```

my_list = list(my_tuple)
print(my_list)  # Output: [1, 2, 2, 3]
# Convert to set (removes duplicates, unordered)
my_set = set(my_tuple)
print(my_set)   # Output: {1, 2, 3}

```

